PYAI · MARCH 10, 2026 · SAN FRANCISCO

Leaning In to Find Out

ARMIN RONACHER · CO-FOUNDER, EARENDIL

Who Am I?

- Armin Ronacher, co-founder at Earendil
- Spent 10 years making Sentry a great product
- Created a lot of Open Source software: Flask, Jinja2, and others
- I write at `lucumr.pocoo.org` about agentic engineering

I Am Going to Speculate a Little Here.

But Hopefully in a Useful Direction.

We Are All Somewhat Beholden to Foundation Model Providers.

THAT MEANS WE HAVE TO LEARN FROM BEHAVIOR, NOT FROM INTERNALS.

ARMIN RONACHER

EARENDIL INC.

The Weird Part

- As user we do not control the training process
- We rarely know what the models were optimized for
- What if the models change and our uses perform badly?

So the Job Is Not to Guess.
It Is to Find Out.

To Find Out
With a Way to Extrapolate.

ARMIN RONACHER

EARENDIL INC.

Understand Reinforcement Learning

- A pretrained model gives you a broad prior
- RL then samples behavior and attaches a reward signal
- Good trajectories get reinforced, bad ones get suppressed
- Over time, the model becomes more like the behavior that keeps scoring well

What Do the Model Providers Train On?

IN SHORT: ENTIRE SESSIONS THEY GET ACCESS TO.

- prompts, responses, tool calls, edits, retries, tests
- ranking signals: task success, acceptance, abandonment
- particularly things like Claude Code sessions where the loop is rich
- As we vibe, we reinforce

Why That Matters

Models improve fastest where people use them a lot and the outcome is easy to judge.

- High-volume successful workflows create pressure to improve there
- Tasks with clear evaluators are easier to reinforce aggressively
- Tool-using coding loops are unusually measurable

So How Do We Extrapolate?

- Do not just ask what is popular
- Ask where behavior is abundant and legible
- Ask where outcomes can be scored cheaply
- Ask which environments dominate those traces: shells, files, test loops

Gains from Coding Agents

OVERREPRESENTED IN CODING TRACES

- Unix and shell workflows
- files, paths, diffs, permissions
- popular languages and build tools
- tests, logs, stack traces
- file system navigation and incremental edits

UNDERREPRESENTED

- GUI-heavy interaction
- rare proprietary systems
- domains with little trace volume
- tasks that resist textual decomposition

Bonus Hunches

- Coding agents increasingly bring screenshots into the loop
- That means visual understanding in the context of code or code driven agent turns
- And coding-adjacent tooling is becoming the base layer for workspace automation

Layers Above the Coding Substrate

- The same agents now manipulate spreadsheets, Word documents, calendars, and similar workspace surfaces
- They are also remote controlling `tmux` sessions and web browsers via CDP
- So the coding-agent substrate is starting to absorb more and more of ordinary computer work

Surprises

- Curious failure modes show up when important state changes are not visible in the session transcript
- Agents often get very confused when state changes outside the turn and the harness does not inject those changes
- If you tell them about a change and ask them to validate it, they often hallucinate and claim they called a tool when they did not

What This Means for Us

- Do not just optimize for what the model can do today
- Build around workflows with dense textual feedback
- Expect the substrate of coding to keep improving: writing code, running it, files
- That gives you a strategy that survives model drift for a while

Narrowing In on Coding Agents

ARMIN RONACHER

EARENDIL INC.

Coding Is a Mode of Execution

- If the file system works, and agents know how to program, you can route a surprising amount of work through that loop
- Many "non-coding" tasks become tractable once they look like files, programs, tools, and outputs
- That gives you leverage from the model's strongest priors instead of fighting them

Build Tiny Programmable Worlds

- Use simple sandboxed languages with basic sandboxes
- Prefer text files, explicit inputs, deterministic outputs, and good error messages
- If the agent can iterate by editing, running, and reading failures, you inherit the coding-agent gradient

Alignment Beats Novelty

- `SQL` beats a custom query DSL
- `JavaScript light` beats a complicated expression language
- Even a bespoke DSL can work if it aligns with a known language and produces excellent errors
- The more it feels like programming, the more capability you get for free

Finding Out

SO NOW THAT WE HAVE A HYPOTHESIS, HOW DO WE FIND OUT IF IT'S RIGHT?

ARMIN RONACHER

EARENDIL INC.

Initial Pass, Then Iteration

- Ask the model a few times how it would do the thing
- Models have some weak introspectability about likely strategies and failure modes
- Then feed agentic turns back into models so they can critique the interaction and suggest harness, prompt, and tool improvements

One Practical Loop

- Run the same basic task twenty times with a coding agent
- Use subagents or small variations if that helps you explore the space
- Then let another LLM score the outcomes for quality, failure modes, and consistency

Use Your Own Traces

- Your own coding sessions are a valuable input for judgement
- Things like `.{pi,claude,codex}/sessions` accumulate a surprising amount of useful data
- At the beginning, reduce variables: fewer tools, simpler harnesses, clearer prompts

The Trick Is Not Just to Find Out.
It Is to Find Out in a Way That Extrapolates.